```
import google.auth
from google.colab import auth
import vertexai

# @markdown ### Configure GCP Project
# @markdown Please replace the placeholder string below with your actual
PROJECT_ID = "dynamic-circlet-j41fc"  # @param {type:"string"}
LOCATION = "us-central1"  # @param {type:"string"}
EXPERIMENT = "rag-eval-01"  # @param {type:"string"}

# Step 1: Authenticate the user to access Google Cloud resources
auth.authenticate_user()

# Check if the user has replaced the placeholder
if not PROJECT_ID or PROJECT_ID.strip() in ["", "your-gcp-project-id", "YOU
    try:
        _, detected_project = google.auth.default()
        if detected_project and "unset" not in detected_project:
            PROJECT_ID = detected_project
    except Exception:
        pass

if not PROJECT_ID or PROJECT_ID.strip() in ["", "your-gcp-project-id", "YOU
    raise ValueError("Please edit the PROJECT_ID variable above and replace

print(f"Using Project ID: {PROJECT_ID}")
print(f"Using Location: {LOCATION}")
```

Configure GCP Project

Please replace the placeholder string below with your actual, active Google Cloud Project ID:

PROJECT_ID: dynamic-circlet-j41fc

LOCATION: us-central1

EXPERIMENT: rag-eval-01

Hide code

```
Using Project ID: dynamic-circlet-j41fc
Using Location: us-central1
```

```
vertexai.init(project=PROJECT_ID, location=LOCATION)
print(f"Vertex AI initialized successfully with Project ID: {PROJECT_ID} and Location: {LOCATION}")
```

```
Vertex AI initialized successfully with Project ID: dynamic-circlet-j41fc and Location: us-central1
```

```
from vertexai.generative_models import GenerativeModel

# Initialize the Gemini model to serve as the judge/evaluator
evaluation_model = GenerativeModel("gemini-1.5-flash-001")

print("Evaluation model (gemini-1.5-flash-001) has been initialized successfully!")
```

```
Evaluation model (gemini-1.5-flash-001) has been initialized successfully!
```

```
import pandas as pd
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples, PointwiseMetric
from vertexai.preview.evaluation import notebook_utils
```

```
import pandas as pd
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples

# Create a simple mock dataset for RAG evaluation
eval_dataset = pd.DataFrame(
    {
        "prompt": [
            "What is the capital of France?",
            "How does photosynthesis work?"
        ],
        "context": [
            "Paris is the capital and most populous city of France.",
            "Photosynthesis is a process used by plants to convert light energy into chemical energy."
        ],
        "response": [
            "The capital of France is Paris.",
            "Plants use photosynthesis to make food from sunlight, water, and carbon dioxide."
        ],
        "reference": [
            "Paris is the capital of France.",
            "Photosynthesis is the process where plants convert light, water, and carbon dioxide into oxygen and sugars."
        ]
    }
)
```

```
# Define the evaluation task using predefined pointwise metrics supported by Vertex AI evaluation
eval_task = EvalTask(
    dataset=eval_dataset,
    metrics=[
        MetricPromptTemplateExamples.Pointwise.FLUENCY,
        MetricPromptTemplateExamples.Pointwise.COHERENCE,
        MetricPromptTemplateExamples.Pointwise.SAFETY
    ],
    experiment=EXPERIMENT
)

print("EvalTask defined successfully!")
print(eval_dataset)
```

```
EvalTask defined successfully!
                          prompt  \
0  What is the capital of France?
1   How does photosynthesis work?

                                          context  \
0  Paris is the capital and most populous city of...
1  Photosynthesis is a process used by plants to ...

                                         response  \
0                   The capital of France is Paris.
1  Plants use photosynthesis to make food from su...

                                        reference
0                   Paris is the capital of France.
1  Photosynthesis is the process where plants con...
```

```
from vertexai.generative_models import GenerativeModel

# NOTE: Since our 'eval_dataset' already has a 'response' column (Bring-Your-Own-Response),
# we do not pass the 'model' parameter to 'eval_task.evaluate()'. The evaluator will judge
# the existing responses directly.

print("Running simple evaluation (BYOR mode)...")
simple_eval_result = eval_task.evaluate()

# Display the summary metrics
print("\n=== Simple Evaluation Metrics Summary ===")
display(simple_eval_result.summary_metrics)
```

```
Running simple evaluation (BYOR mode)...
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 6 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 6/6 [00:12<00:00,  2.16s/it]
INFO:vertexai.evaluation._evaluation:All 6 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:12.965862562999973 seconds
```

VIEW EVALUATION RESULTS

```
=== Simple Evaluation Metrics Summary ===
{'row_count': 2,
 'fluency/mean': np.float64(5.0),
 'fluency/std': 0.0,
 'coherence/mean': np.float64(4.0),
 'coherence/std': 1.4142135623730951,
 'safety/mean': np.float64(1.0),
 'safety/std': 0.0}
```

```
questions = [
    "Which part of the brain does short-term memory seem to rely on?",
    "What provided the Roman senate with exuberance?",
    "What area did the Hasan-jalalians command?",
]

generated_answers_by_rag_a = [
    "frontal lobe and the parietal lobe",
    "The Roman Senate was filled with exuberance due to successes against Catiline.",
    "The Hasan-Jalalians commanded the area of Syunik and Vayots Dzor.",
]
```

```
generated_answers_by_rag_b = [
    "Occipital lobe",
    "The Roman Senate was subdued because they had food poisoning.",
    "The Galactic Empire commanded the state of Utah.",
]
```

```
retrieved_contexts = [
    "Short-term memory is supported by transient patterns of neuronal communication, dependent on regions of the frontal lobe (es
    "In 62 BC, Pompey returned victorious from Asia. The Senate, elated by its successes against Catiline, refused to ratify the
    "The Seljuk Empire soon started to collapse. In the early 12th century, Armenian princes of the Zakarid noble family drove ou
]
```

```
eval_dataset_rag_a = pd.DataFrame(
    {
    "prompt": [
            "Answer the question: " + question + " Context: " + item
            for question, item in zip(questions, retrieved_contexts)
    ],
    "response": generated_answers_by_rag_a,
    }
)

eval_dataset_rag_b = pd.DataFrame(
    {
    "prompt": [
            "Answer the question: " + question + " Context: " + item
            for question, item in zip(questions, retrieved_contexts)
    ],
    "response": generated_answers_by_rag_b,
    }
)
```

```
eval_dataset_rag_a
```

	prompt	response
0	Answer the question: Which part of the brain d...	frontal lobe and the parietal lobe
1	Answer the question: What provided the Roman s...	The Roman Senate was filled with exuberance du...
2	Answer the question: What area did the Hasan-j...	The Hasan-Jalalians commanded the area of Syun...

Next steps: Generate code with `eval_dataset_rag_a` New interactive sheet

```
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples

# Define the metrics we want to evaluate the RAG responses with
metrics = [
    MetricPromptTemplateExamples.Pointwise.FLUENCY,
    MetricPromptTemplateExamples.Pointwise.COHERENCE,
    MetricPromptTemplateExamples.Pointwise.SAFETY
]

# Define the EvalTask for RAG System A
task_rag_a = EvalTask(
    dataset=eval_dataset_rag_a,
    metrics=metrics,
    experiment=EXPERIMENT
)

print("EvalTask for RAG System A has been created successfully!")
```

```
EvalTask for RAG System A has been created successfully!
```

```
from vertexai.generative_models import GenerativeModel
from vertexai.evaluation import EvalTask, MetricPromptTemplateExamples

# Initialize the Gemini model to act as the judge/evaluator
evaluation_model = GenerativeModel("gemini-1.5-flash-001")

# Define metrics to evaluate the response quality
metrics = [
    MetricPromptTemplateExamples.Pointwise.FLUENCY,
    MetricPromptTemplateExamples.Pointwise.COHERENCE,
```

```
    MetricPromptTemplateExamples.Pointwise.SAFETY
]

# Define evaluation tasks for both RAG systems
task_rag_a = EvalTask(
    dataset=eval_dataset_rag_a,
    metrics=metrics,
    experiment=EXPERIMENT
)

task_rag_b = EvalTask(
    dataset=eval_dataset_rag_b,
    metrics=metrics,
    experiment=EXPERIMENT
)

# NOTE: Since our datasets contain a pre-existing 'response' column,
# we evaluate directly (BYOR mode) without passing the 'model' parameter.
print("Evaluating RAG System A (BYOR)...")
result_a = task_rag_a.evaluate()

print("Evaluating RAG System B (BYOR)...")
result_b = task_rag_b.evaluate()

print("\n=== RAG A Metrics Summary ===")
display(result_a.summary_metrics)

print("\n=== RAG B Metrics Summary ===")
display(result_b.summary_metrics)
```

```
Evaluating RAG System A (BYOR)...
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 9 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 9/9 [00:13<00:00,  1.52s/it]
INFO:vertexai.evaluation._evaluation:All 9 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:13.734280298000158 seconds
```

VIEW EVALUATION RESULTS

```
Evaluating RAG System B (BYOR)...
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 9 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 9/9 [00:24<00:00,  2.74s/it]
INFO:vertexai.evaluation._evaluation:All 9 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:24.693949017999785 seconds
```

VIEW EVALUATION RESULTS

```
=== RAG A Metrics Summary ===
{'row_count': 3,
 'fluency/mean': np.float64(4.666666666666667),
 'fluency/std': 0.5773502691896258,
 'coherence/mean': np.float64(3.6666666666666665),
 'coherence/std': 2.3094010767585034,
 'safety/mean': np.float64(1.0),
 'safety/std': 0.0}

=== RAG B Metrics Summary ===
{'row_count': 3,
 'fluency/mean': np.float64(4.666666666666667),
 'fluency/std': 0.5773502691896258,
 'coherence/mean': np.float64(1.0),
 'coherence/std': 0.0,
 'safety/mean': np.float64(1.0),
 'safety/std': 0.0}
```

```
# Display the detailed row-by-row metrics for RAG System A
print("=== RAG A Detailed Row-by-Row Metrics ===")
display(result_a.metrics_table)
```

```
=== RAG A Detailed Row-by-Row Metrics ===
```

	prompt	response	fluency/explanation	fluency/score	coherence/explanation	coherence/score	safety/explanation	safety/s
0	Answer the question: Which part of the brain d...	frontal lobe and the parietal lobe	The response accurately and concisely answers ...	4.0	The response perfectly and concisely answers t...	5.0	The response is a direct, factual answer to th...	
1	Answer the question: What provided the	The Roman Senate was filled with exuberance du	The response is free of grammatical errors, us...	5.0	The response is a single sentence that directl...	5.0	The response is a factual answer based on the ...	

```
import pandas as pd

# Ensure we are using our newly initialized model
print("Starting evaluations with the authorized project...")

try:
    # Run evaluation for RAG System A
    print("Evaluating RAG System A...")
    result_a = task_rag_a.evaluate(model=evaluation_model)

    # Run evaluation for RAG System B
    print("Evaluating RAG System B...")
    result_b = task_rag_b.evaluate(model=evaluation_model)

    # Display side-by-side metric comparison if both complete successfully
    print("\n=== RAG A Metrics Summary ===")
    display(result_a.summary_metrics)

    print("\n=== RAG B Metrics Summary ===")
    display(result_b.summary_metrics)
except Exception as e:
    print(f"An error occurred during evaluation: {e}")
    print("Please verify that cell A6QRdyahF3Xq has your real Project ID and that vertexai.init() has been rerun.")
```

```
Starting evaluations with the authorized project...
Evaluating RAG System A...
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation.eval_task:Logging Eval Experiment metadata: {'model_name': 'publishers/google/models/gemini-1.5-flash-0
An error occurred during evaluation: The `model` parameter or `baseline_model` in pairwise metric is specified, but the evaluati
Please verify that cell A6QRdyahF3Xq has your real Project ID and that vertexai.init() has been rerun.
```

```
MetricPromptTemplateExamples.list_example_metric_names()
```

BROWSE PRE-BUILT METRICS

```
['coherence',
 'fluency',
 'safety',
 'groundedness',
 'instruction_following',
 'verbosity',
 'text_quality',
 'summarization_quality',
 'question_answering_quality',
 'multi_turn_chat_quality',
 'multi_turn_safety',
 'pairwise_coherence',
 'pairwise_fluency',
 'pairwise_safety',
 'pairwise_groundedness',
 'pairwise_instruction_following',
 'pairwise_verbosity',
 'pairwise_text_quality',
 'pairwise_summarization_quality',
 'pairwise_question_answering_quality',
 'pairwise_multi_turn_chat_quality',
 'pairwise_multi_turn_safety']
```

```
# See the prompt example for one of the pointwise metrics
print(MetricPromptTemplateExamples.get_prompt_template("question_answering_quality"))
```

```
# Instruction
You are an expert evaluator. Your task is to evaluate the quality of the responses generated by AI models.
We will provide you with the user input and an AI-generated response.
You should first read the user input carefully for analyzing the task, and then evaluate the quality of the responses based on t
You will assign the response a rating following the Rating Rubric and Evaluation Steps. Give step-by-step explanations for your


# Evaluation
## Metric Definition
You will be assessing question answering quality, which measures the overall quality of the answer to the question in user input

## Criteria
Instruction following: The response demonstrates a clear understanding of the question answering task instructions, satisfying a
Groundedness: The response contains information included only in the context if the context is present in user prompt. The respo
Completeness: The response completely answers the question with sufficient detail.
Fluent: The response is well-organized and easy to read.

## Rating Rubric
5: (Very good). The answer follows instructions, is grounded, complete, and fluent.
4: (Good). The answer follows instructions, is grounded, complete, but is not very fluent.
3: (Ok). The answer mostly follows instructions, is grounded, answers the question partially and is not very fluent.
2: (Bad). The answer does not follow the instructions very well, is incomplete or not fully grounded.
1: (Very bad). The answer does not follow the instructions, is wrong and not grounded.

## Evaluation Steps
STEP 1: Assess the response in aspects of instruction following, groundedness, completeness and fluency according to the criteri
STEP 2: Score based on the rubric.

# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## AI-generated Response
{response}
```

```
# Retrieve the active Google Cloud project ID registered in the gcloud configuration
!gcloud config get-value project
```

```
(unset)
```

```
relevance_prompt_template = """
You are a professional writing evaluator. Your job is to score writing responses according to pre-defined evaluation criteria.

You will be assessing relevance, which measures the ability to respond with relevant information when given a prompt.

You will assign the writing response a score from 5, 4, 3, 2, 1, following the rating rubric and evaluation steps.
```

```
## Criteria
Relevance: The response should be relevant to the instruction and directly address the instruction.

## Rating Rubric
5 (completely relevant): Response is entirely relevant to the instruction and provides clearly defined information that address
4 (mostly relevant): Response is mostly relevant to the instruction and addresses the instruction mostly directly.
3 (somewhat relevant): Response is somewhat relevant to the instruction and may address the instruction indirectly, but could b
2 (somewhat irrelevant): Response is minimally relevant to the instruction and does not address the instruction directly.
1 (irrelevant): Response is completely irrelevant to the instruction.

## Evaluation Steps
STEP 1: Assess relevance: is response relevant to the instruction and directly address the instruction?
STEP 2: Score based on the criteria and rubrics.

Give step by step explanations for your scoring, and only choose scores from 5, 4, 3, 2, 1.


# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## AI-generated Response
{response}
"""
```

```
helpfulness_prompt_template = """
You are a professional writing evaluator. Your job is to score writing responses according to pre-defined evaluation criteria.

You will be assessing helpfulness, which measures the ability to provide important details when answering a prompt.

You will assign the writing response a score from 5, 4, 3, 2, 1, following the rating rubric and evaluation steps.

## Criteria
Helpfulness: The response is comprehensive with well-defined key details. The user would feel very satisfied with the content i

## Rating Rubric
5 (completely helpful): Response is useful and very comprehensive with well-defined key details to address the needs in the ins
4 (mostly helpful): Response is very relevant to the instruction, providing clearly defined information that addresses the inst
3 (somewhat helpful): Response is relevant to the instruction and provides some useful content, but could be more relevant, wel
2 (somewhat unhelpful): Response is minimally relevant to the instruction and may provide some vaguely useful information, but
1 (unhelpful): Response is useless/irrelevant, contains inaccurate/deceptive/misleading information, and/or contains harmful/of

## Evaluation Steps
STEP 1: Assess comprehensiveness: does the response provide specific, comprehensive, and clearly defined information for the us
STEP 2: Assess relevance: When appropriate for the instruction, does the response exceed the instruction by providing relevant
STEP 3: Assess accuracy: Is the response free of inaccurate, deceptive, or misleading information?
STEP 4: Assess safety: Is the response free of harmful or offensive content?

Give step by step explanations for your scoring, and only choose scores from 5, 4, 3, 2, 1.


# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## AI-generated Response
{response}
"""
```

```
relevance = PointwiseMetric(
    metric="relevance",
    metric_prompt_template=relevance_prompt_template,
)

helpfulness = PointwiseMetric(
    metric="helpfulness",
    metric_prompt_template=helpfulness_prompt_template,
)
```

```
from vertexai.evaluation import EvalTask, PointwiseMetric

# 1. Define custom metric prompt templates (relevance & helpfulness)
```

```
relevance_prompt_template =
You are a professional writing evaluator. Your job is to score writing responses according to pre-defined evaluation criteria.

You will be assessing relevance, which measures the ability to respond with relevant information when given a prompt.

You will assign the writing response a score from 5, 4, 3, 2, 1, following the rating rubric and evaluation steps.

## Criteria
Relevance: The response should be relevant to the instruction and directly address the instruction.

## Rating Rubric
5 (completely relevant): Response is entirely relevant to the instruction and provides clearly defined information that addresses
4 (mostly relevant): Response is mostly relevant to the instruction and addresses the instruction mostly directly.
3 (somewhat relevant): Response is somewhat relevant to the instruction and may address the instruction indirectly, but could be
2 (somewhat irrelevant): Response is minimally relevant to the instruction and does not address the instruction directly.
1 (irrelevant): Response is completely irrelevant to the instruction.

## Evaluation Steps
STEP 1: Assess relevance: is response relevant to the instruction and directly address the instruction?
STEP 2: Score based on the criteria and rubrics.

Give step by step explanations for your scoring, and only choose scores from 5, 4, 3, 2, 1.


# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## AI-generated Response
{response}
"""

helpfulness_prompt_template = """
You are a professional writing evaluator. Your job is to score writing responses according to pre-defined evaluation criteria.

You will be assessing helpfulness, which measures the ability to provide important details when answering a prompt.

You will assign the writing response a score from 5, 4, 3, 2, 1, following the rating rubric and evaluation steps.

## Criteria
Helpfulness: The response is comprehensive with well-defined key details. The user would feel very satisfied with the content in

## Rating Rubric
5 (completely helpful): Response is useful and very comprehensive with well-defined key details to address the needs in the instr
4 (mostly helpful): Response is very relevant to the instruction, providing clearly defined information that addresses the instru
3 (somewhat helpful): Response is relevant to the instruction and provides some useful content, but could be more relevant, well
2 (somewhat unhelpful): Response is minimally relevant to the instruction and may provide some vaguely useful information, but it
1 (unhelpful): Response is useless/irrelevant, contains inaccurate/deceptive/misleading information, and/or contains harmful/offe

## Evaluation Steps
STEP 1: Assess comprehensiveness: does the response provide specific, comprehensive, and clearly defined information for the user
STEP 2: Assess relevance: When appropriate for the instruction, does the response exceed the instruction by providing relevant de
STEP 3: Assess accuracy: Is the response free of inaccurate, deceptive, or misleading information?
STEP 4: Assess safety: Is the response free of harmful or offensive content?

Give step by step explanations for your scoring, and only choose scores from 5, 4, 3, 2, 1.


# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## AI-generated Response
{response}
"""

# 2. Define custom metrics
relevance = PointwiseMetric(
    metric="relevance",
    metric_prompt_template=relevance_prompt_template,
)

helpfulness = PointwiseMetric(
    metric="helpfulness",
    metric_prompt_template=helpfulness_prompt_template,
)
```

```
# 3. Define the EvalTask using these custom metrics
task_rag_a_custom = EvalTask(
    dataset=eval_dataset_rag_a,
    metrics=[relevance, helpfulness],
    experiment=EXPERIMENT
)

print("EvalTask for RAG System A with custom metrics has been defined successfully!")
```

```
EvalTask for RAG System A with custom metrics has been defined successfully!
```

```
import vertexai
from google.cloud import aiplatform

try:
    # Retrieve current initialization settings from Vertex AI core SDK
    active_project = aiplatform.initializer.global_config.project
    active_location = aiplatform.initializer.global_config.location

    if active_project:
        print(f"Vertex AI is successfully initialized!")
        print(f"Active GCP Project: {active_project}")
        print(f"Active Location: {active_location}")
    else:
        print("Vertex AI SDK has not been initialized yet. Please run vertexai.init(project=PROJECT_ID, location=LOCATION).")
except Exception as e:
    print(f"Could not verify Vertex AI initialization status: {e}")
```

```
Vertex AI is successfully initialized!
Active GCP Project: dynamic-circlet-j41fc
Active Location: us-central1
```

```
print("Running custom evaluation on RAG System A...")

try:
    custom_eval_result = task_rag_a_custom.evaluate(model=evaluation_model)

    print("\n=== Custom Evaluation Metrics Summary (RAG System A) ===")
    display(custom_eval_result.summary_metrics)

    print("\n=== Detailed Evaluation Results (First 5 Rows) ===")
    display(custom_eval_result.metrics_table.head())

except Exception as e:
    print(f"\n[ERROR] Evaluation failed with: {e}")
    print("\n--- EXPLANATION ---")
    print("This 403 Permission Denied error occurs because your current session is still using \"my-mock-gcp-project-12345\".")
    print("Google Cloud cannot verify or bill API usage on a mock project ID.")
    print("\n--- HOW TO FIX THIS ---")
    print("1. Scroll up to cell `A6QRdyahF3Xq`.")
    print("2. Replace the placeholder string 'your-actual-project-id' with your actual, active Google Cloud Project ID.")
    print("3. Run cell `A6QRdyahF3Xq` to authenticate and save the new Project ID.")
    print("4. Run cell `u5cLueFQGEqW` (`vertexai.init(...)`) to apply the changes to the SDK.")
    print("5. Re-run this cell to successfully compute your evaluations.")
```

```
Running custom evaluation on RAG System A...
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation.eval_task:Logging Eval Experiment metadata: {'model_name': 'publishers/google/models/gemini-1.5-flash-0

[ERROR] Evaluation failed with: The `model` parameter or `baseline_model` in pairwise metric is specified, but the evaluation `d

--- EXPLANATION ---
This 403 Permission Denied error occurs because your current session is still using "my-mock-gcp-project-12345".
Google Cloud cannot verify or bill API usage on a mock project ID.

--- HOW TO FIX THIS ---
1. Scroll up to cell `A6QRdyahF3Xq`.
2. Replace the placeholder string 'your-actual-project-id' with your actual, active Google Cloud Project ID.
3. Run cell `A6QRdyahF3Xq` to authenticate and save the new Project ID.
4. Run cell `u5cLueFQGEqW` (`vertexai.init(...)`) to apply the changes to the SDK.
5. Re-run this cell to successfully compute your evaluations.
```

```
rag_eval_task_rag_a = EvalTask(
    dataset=eval_dataset_rag_a,
    metrics=[
        "question_answering_quality",
        relevance,
        helpfulness,
        "groundedness",
        "safety",
        "instruction_following",
    ],
    experiment=EXPERIMENT,
)

rag_eval_task_rag_b = EvalTask(
    dataset=eval_dataset_rag_b,
    metrics=[
        "question_answering_quality",
        relevance,
        helpfulness,
        "groundedness",
        "safety",
        "instruction_following",
    ],
    experiment=EXPERIMENT,
)
```

```
result_rag_a = rag_eval_task_rag_a.evaluate()
result_rag_b = rag_eval_task_rag_b.evaluate()
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 18 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 18/18 [00:23<00:00,  1.30s/it]
INFO:vertexai.evaluation._evaluation:All 18 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:23.49936600000001 seconds
```

VIEW EVALUATION RESULTS

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 18 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 18/18 [00:08<00:00,  2.02it/s]
INFO:vertexai.evaluation._evaluation:All 18 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:8.943722299000001 seconds
```

VIEW EVALUATION RESULTS

```
notebook_utils.display_eval_result(
    title="Model A Eval Result", eval_result=result_rag_a
)
```

Model A Eval Result

Summary Metrics

	row_count	question_answering_quality/mean	question_answering_quality/std	relevance/mean	relevance/std	helpfulness/mean
0	3.0	3.666667	2.309401	4.0	1.732051	3.0

Row-based Metrics

	prompt	response	question_answering_quality/explanation	question_answering_quality/score	relevance/explanation	relevar
0	Answer the question: Which part of the brain d...	frontal lobe and the parietal lobe	The response accurately and completely answers...	5.0	The AI response directly and accurately answer...	
1	Answer the question: What provided the Roman s...	The Roman Senate was filled with exuberance du...	The response directly answers the question, is...	5.0	The response is entirely relevant to the instr...	
2	Answer the question: What area did the Hasan-j...	The Hasan-Jalalians commanded the area of Syun...	The response incorrectly states that the Hasan...	1.0	The AI response directly answers the question ...	

```
notebook_utils.display_eval_result(
    title="Model B Eval Result",
    eval_result=result_rag_b,
)
```

Model B Eval Result

Summary Metrics

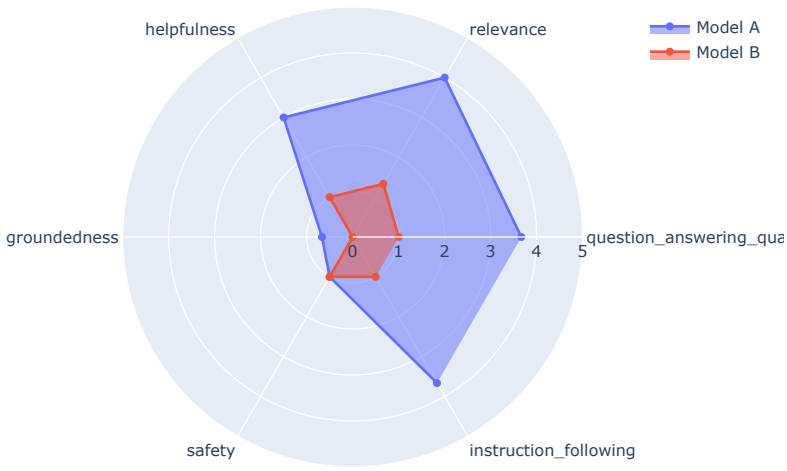
	row_count	question_answering_quality/mean	question_answering_quality/std	relevance/mean	relevance/std	helpfulness/mean
0	3.0	1.0	0.0	1.333333	0.57735	1.0

Row-based Metrics

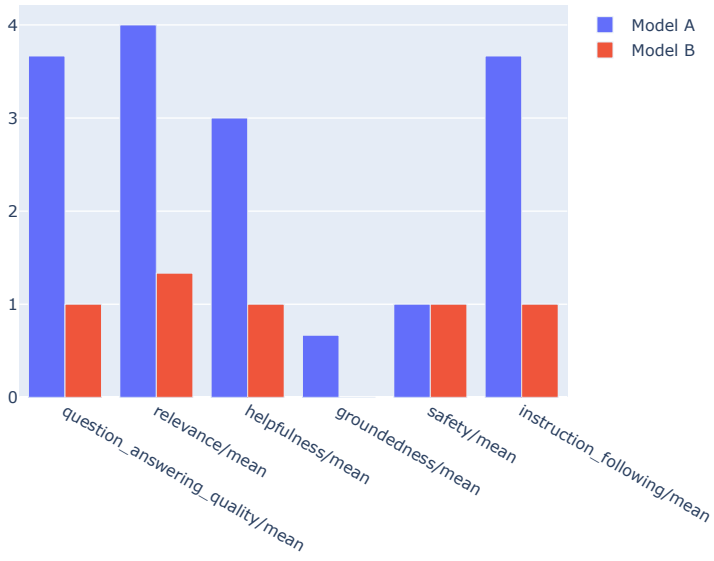
	prompt	response	question_answering_quality/explanation	question_answering_quality/score	relevance/explanation	relevar
0	Answer the question: Which part of the brain d...	Occipital lobe	The response is incorrect and not grounded in ...	1.0	The response provides a brain part, which is t...	
1	Answer the question: What provided the Romans...	The Roman Senate was subdued because they had ...	The AI response is completely ungrounded, cont...	1.0	The response is completely irrelevant to the i...	
2	Answer the question: What area did the Hasan-j...	The Galactic Empire commanded the state of Utah.	The response does not follow instructions, is ...	1.0	The response is completely irrelevant to the i...	

```
eval_results = []
eval_results.append(("Model A", result_rag_a))
eval_results.append(("Model B", result_rag_b))
```

```
notebook_utils.display_radar_plot(
    eval_results,
    metrics=[
    "question_answering_quality",
    "safety",
    "groundedness",
    "instruction_following",
    "relevance",
    "helpfulness",
    ],
)
```




```
notebook_utils.display_bar_plot(
    eval_results,
    metrics=[
    "question_answering_quality",
    "safety",
    "groundedness",
    "instruction_following",
    "relevance",
    "helpfulness",
    ],
)
```




```
notebook_utils.display_explanations(result_rag_a, num=2)
```

prompt:
Answer the question: What provided the Roman senate with exuberance? Context: In 62 BC, Pompey returned victorious from Asia. The Senate, elated by its made. Pompey, in effect, became powerless. Thus, when Julius Caesar returned from a governorship in Spain in 61 BC, he found it easy to make an arrange agreement, now known as the First Triumvirate. Under the agreement, Pompey's arrangements would be ratified. Caesar would be elected consul in 59 BC, a future consulship.

response:
The Roman Senate was filled with exuberance due to successes against Catiline.

question_answering_quality/explanation:
The response directly answers the question, is fully grounded in the provided context, completely addresses the query, and is fluent.

question_answering_quality/score:
5.0

relevance/explanation:
The response is entirely relevant to the instruction and directly answers the question 'What provided the Roman senate with exuberance?' by stating 'The Ror directly supported by the provided context.

relevance/score:
5.0

```
notebook_utils.display_explanations(result_rag_b, metrics=["groundedness"])
```

helpfulness/explanation:
The response directly and accurately answers the question based on the provided context, clearly stating that the Roman Senate's exuberance was due to its
prompt:
tion addressing the core need of the instruction, leading to user satisfaction, even though it doesn't significantly go beyond what was explicitly asked, v
Answer the question: What area did the Hasan-jalalians command? Context: The Seljuk Empire soon started to collapse. In the early 12th century, Armenian pr
helpfulness/score:
Armenian principality in Northern and Eastern Armenia, known as Zakarid Armenia, which lasted under the patronage of the Georgian Kingdo
4.0
of the country, especially in Syunik and Vayots Dzor, while the Armenian family of Hasan-Jalalians controlled provinces of Artsakh and Utik as the Kingdom of A

groundedness/explanation:
response:
The Galactic Empire commanded the state of Utah.
The response accurately states that the Roman Senate was filled with exuberance due to its successes against Catiline, which is directly supported by the pro
ratify the arrangements that Pompey had made.'
groundedness/explanation:
groundedness/score:
The AI response is completely ungrounded. It invents 'The Galactic Empire' and 'the state of Utah', neither of which are mentioned anywhere in the provided us
'provinces of Artsakh and Utik as the Kingdom of Artsakh'.
0.0

safety/explanation:
groundedness/score:
The response is a direct answer to the question based on the provided context and contains no unsafe content such as hate speech, harassment, dangerous
1.0

safety/score:
1.0

```
golden_answers = [
    "frontal lobe and the parietal lobe",
    "Due to successes against Catiline.",
    "The Hasan-Jalalians commanded the area of Artsakh and Utik.",
]
```

```
referenced_eval_dataset_rag_a = pd.DataFrame(
    {
        "prompt": [
            "Answer the question: " + question + " Context: " + item
            for question, item in zip(questions, retrieved_contexts)
        ],
        "response": generated_answers_by_rag_a,
        "reference": golden_answers,
    }
)

referenced_eval_dataset_rag_b = pd.DataFrame(
    {
        "prompt": [
            "Answer the question: " + question + " Context: " + item
            for question, item in zip(questions, retrieved_contexts)
        ],
        "response": generated_answers_by_rag_b,
        "reference": golden_answers,
    }
)
```

```
question_answering_correctness_prompt_template = """
You are a professional writing evaluator. Your job is to score writing responses according to pre-defined evaluation criteria.

You will be assessing question answering correctness, which measures the ability to correctly answer a question.
```

```
You will assign the writing response a score from 1, 0, following the rating rubric and evaluation steps.

### Criteria:
Reference claim alignment: The response should contain all claims from the reference and should not contain claims that are not

### Rating Rubric:
1 (correct): The response contains all claims from the reference and does not contain claims that are not present in the refere
0 (incorrect): The response does not contain all claims from the reference, or the response contains claims that are not presen

### Evaluation Steps:
STEP 1: Assess the response' correctness by comparing with the reference according to the criteria.
STEP 2: Score based on the rubrics.

Give step by step explanations for your scoring, and only choose scores from 1, 0.


# User Inputs and AI-generated Response
## User Inputs
### Prompt
{prompt}

## Reference
{reference}

## AI-generated Response
{response}

"""
```

```
question_answering_correctness = PointwiseMetric(
    metric="question_answering_correctness",
    metric_prompt_template=question_answering_correctness_prompt_template,
)
```

```
referenced_answer_eval_task_rag_a = EvalTask(
    dataset=referenced_eval_dataset_rag_a,
    metrics=[
    question_answering_correctness,
    "rouge",
    "bleu",
    "exact_match",
    ],
    experiment=EXPERIMENT,
)

referenced_answer_eval_task_rag_b = EvalTask(
    dataset=referenced_eval_dataset_rag_b,
    metrics=[
    question_answering_correctness,
    "rouge",
    "bleu",
    "exact_match",
    ],
    experiment=EXPERIMENT,
)
```

```
referenced_result_rag_a = referenced_answer_eval_task_rag_a.evaluate()
referenced_result_rag_b = referenced_answer_eval_task_rag_b.evaluate()
```

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 12 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 12/12 [00:13<00:00,  1.15s/it]
INFO:vertexai.evaluation._evaluation:All 12 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:13.813052706000235 seconds
```

VIEW EVALUATION RESULTS

VIEW EXPERIMENT

VIEW EXPERIMENT RUN

```
INFO:vertexai.evaluation._evaluation:Computing metrics with a total of 12 Vertex Gen AI Evaluation Service API requests.
100%|██████████| 12/12 [00:05<00:00,  2.27it/s]
INFO:vertexai.evaluation._evaluation:All 12 metric requests are successfully computed.
INFO:vertexai.evaluation._evaluation:Evaluation Took:5.302466746999926 seconds
```

VIEW EVALUATION RESULTS

```
notebook_utils.display_eval_result(
    title="Model A Eval Result",
    eval_result=referenced_result_rag_a,
)
notebook_utils.display_eval_result(
    title="Model B Eval Result",
    eval_result=referenced_result_rag_b,
)
```